

Processors

The **CPU (Central Processing Unit)** is the brain of the computer. It executes instructions, manages computation using registers, and works closely with the operating system, especially during context switching, system calls, traps, and protection between user mode and kernel mode.

Basic Role of the CPU

The CPU is responsible for fetching instructions from memory and executing them. Programs run because the CPU repeatedly performs a simple cycle:

1. fetch an instruction from memory,
2. decode the instruction,
3. execute it,
4. move to the next instruction.

This cycle continues until the program finishes.

The repeated **fetch** – **decode** – **execute** cycle is the fundamental working principle of every processor.

Instruction Sets and Architecture

Each CPU can execute only its own specific instruction set. For example:

- an **x86** processor cannot directly execute ARM programs,
- an **ARM** processor cannot directly execute x86 programs.

This means software must match the processor architecture it is built for.

Why registers are needed

Memory access is much slower than CPU execution. To improve performance, CPUs contain fast internal storage locations called **registers**.

Registers are used to hold:

- important variables,

- temporary results,
- addresses,
- control information.

So the instruction set usually includes instructions to:

- load data from memory into a register,
- store data from a register back to memory,
- perform operations using register values.

Important CPU Registers

Besides general-purpose registers, CPUs also have several special registers that are important to both programs and the operating system.

Program Counter (PC)

The **Program Counter** contains the memory address of the next instruction to be fetched.

After an instruction is fetched, the program counter is updated to point to the next one.

Stack Pointer (SP)

The **Stack Pointer** points to the top of the current stack in memory.

The stack contains a frame for each active procedure call. A stack frame typically stores:

- input parameters,
- local variables,
- temporary data not kept in registers.

Program Status Word (PSW)

The **PSW (Program Status Word)** stores condition codes, CPU priority, the current mode (user or kernel), and other control bits.

The PSW is especially important for:

- comparisons,
- system calls,
- interrupt and trap handling,
- I/O control,
- protection.

The operating system must know the contents of all important registers, because it has to save and restore them whenever it switches the CPU from one program to another.

CPU and the Operating System

When multiple programs share the CPU, the operating system performs **time multiplexing**. That means:

- one program runs for some time,
- then the OS stops it,
- then another program runs,
- later the first program may continue again.

To do this correctly, the operating system must save the current program's CPU state, including:

- general registers,
- program counter,
- stack pointer,
- status information.

When the program resumes, those values are restored.

Saving and restoring CPU registers during a switch from one program to another is essential for multitasking.

Pipeline CPU

Modern CPUs do not simply execute one instruction fully before starting the next. To improve speed, they use a **pipeline**.

A **pipeline** divides instruction execution into stages, so different instructions can be in different stages at the same time.

Typical stages are:

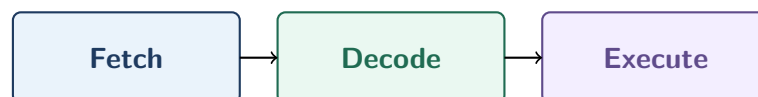
- Fetch
- Decode
- Execute

So while one instruction is being executed:

- the next one may be decoded,
- another one may be fetched.

This increases CPU efficiency.

Simple Pipeline Illustration



Pipelines improve performance, but they also create complications, especially when branches or exceptions occur.

Superscalar CPU

An even more advanced design is the **superscalar CPU**.

A **superscalar CPU** can fetch, decode, and execute multiple instructions at once using multiple execution units.

For example, a CPU may have separate units for:

- integer arithmetic,
- floating-point arithmetic,
- Boolean or logical operations.

Instructions are fetched together, decoded, and placed into a holding buffer. When an execution unit becomes free, it selects a suitable instruction and executes it.

Important consequence

Because of this design:

- instructions may execute **out of order**,
- hardware must ensure the final result remains correct,
- complexity increases for both hardware and the operating system.

CPU Modes: Kernel Mode and User Mode

Most CPUs have two operating modes:

1. **Kernel Mode**
2. **User Mode**

Kernel Mode

In kernel mode:

- the CPU can execute all instructions,
- all hardware features are accessible,
- the operating system normally runs here.

User Mode

In user mode:

- only a subset of instructions can be executed,
- I/O instructions are usually forbidden,
- memory protection features restrict access,
- application programs normally run here.

Kernel mode has full privilege. User mode has limited privilege. This separation protects the system.

System Calls and Trap Mechanism

User programs cannot directly perform privileged operations. To request OS services, they use a **system call**.

A **system call** is a controlled request by a user program to the operating system. It causes a trap into the kernel.

The **TRAP** instruction:

- switches the CPU from user mode to kernel mode,
- transfers control to the operating system,
- allows the OS to perform the requested service.

When the OS finishes, control returns to the user program.

You can think of a system call as:

- a special procedure call,
- plus a mode switch from user mode to kernel mode.

Other Traps and Exceptions

Not all traps come from system calls. Some are caused directly by hardware when exceptional conditions occur, such as:

- division by zero,
- floating-point underflow,
- other exceptional situations.

In every case, the operating system gets control and decides what to do.

Possible actions include:

- terminating the program,
- ignoring or correcting the condition,
- returning control to the program if it has arranged to handle the event.

Traps and exceptions are important because they give the operating system control whenever something unusual or privileged happens.

Multithreaded CPUs

Modern chips often support **multithreading** (also called **hyperthreading** in Intel terminology).

A multithreaded CPU can hold the state of more than one thread and switch between them very quickly.

If one thread is waiting, for example for memory access, the CPU can switch to another thread instead of staying idle.

Important point

Multithreading:

- improves utilization,
- reduces thread-switching delay,
- does **not** provide full parallel execution on a single core.

Only one thread may actually run at a time on that execution path, but switching is very fast.

OS implication

To the operating system, each thread may appear like a separate CPU resource. This can create scheduling issues if the OS places threads inefficiently on the same physical CPU while another CPU stays idle.

Multicore Processors

Beyond multithreading, many chips now contain multiple complete cores.

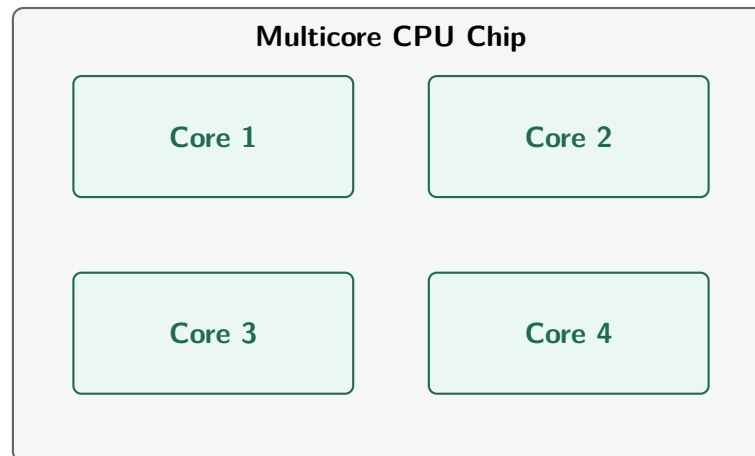
A **multicore processor** has two or more independent CPU cores on a single chip.

Examples:

- quad-core chip = 4 cores,
- many modern chips have 8 or more cores,
- some chips may have dozens of cores.

Each core can execute instructions independently, so multicore chips provide real parallelism.

Simple Multicore Diagram



Multithreading improves switching between threads, while multicore processors provide true parallel execution using multiple cores.

GPU and Many-Core Processing

A **GPU (Graphics Processing Unit)** contains a very large number of small cores.

GPUs are very good for:

- parallel computations,
- graphics rendering,
- workloads with many similar small operations.

They are less suitable for:

- serial tasks,
- complex control-heavy OS logic.

So GPUs may help with special OS-related workloads such as encryption or network processing, but most of the operating system itself typically does not run on the GPU.